MACHINE LEARNING · DECISION TREE

의사결정트리와 랜덤 포레스트

결정 트리 · 지니 불순도 · 엔트로피 · 앙상블 · 랜덤 포레스트

v13 · 2026.10

ⓒ 2026. DJ all rights reserved.

목 차

01 **머신러닝과 결정 트리**
지도학습 안에서 결정 트리의 위치

02 **결정 트리**
노드 · 깊이 · 분류 트리 · 회귀 트리 · 가지치기

03 **결정 트리의 결정 경계**
깊이에 따라 나뉘는 영역

04 **지니 불순도와 엔트로피**
분할 기준 · 계산 예시 · 정보 이득

05 **장단점과 매개변수**
max_depth · min_samples_leaf · 과대적합

06 **앙상블 기법**
배깅 · 부스팅 · 랜덤 포레스트의 아이디어

07 **랜덤 포레스트**
주요 매개변수 · 장단점 · 실습 코드

+ **정리**
한 장으로 보는 결정 트리

ⓒ 2026. DJ all rights reserved.

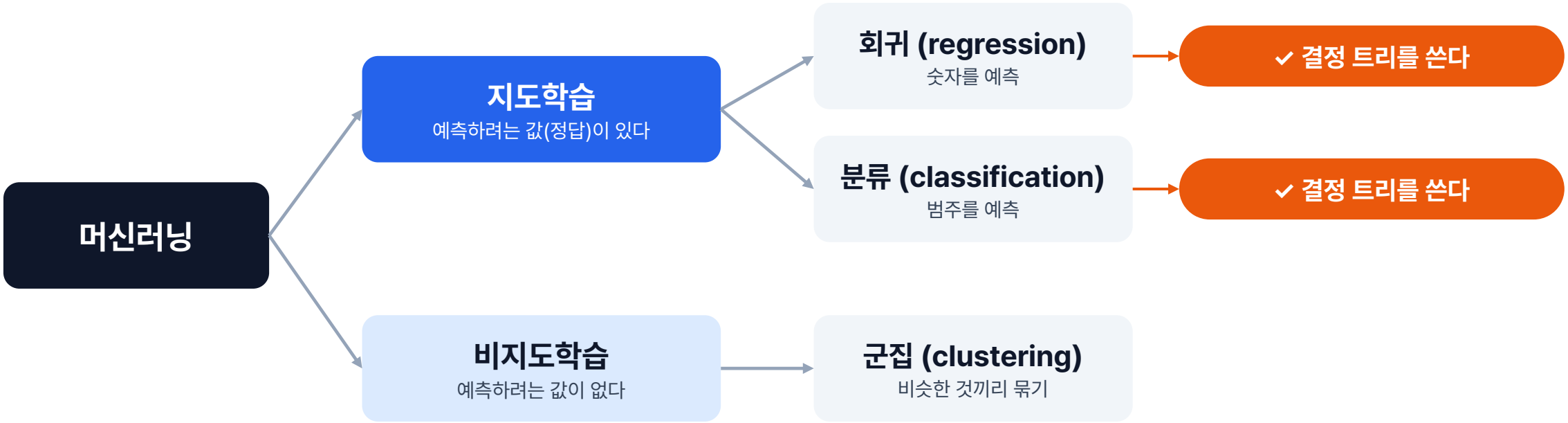
01

머신러닝과 결정 트리

지도학습 안에서 결정 트리는 어디에 있을까

01 머신러닝과 결정 트리

결정 트리는 분류와 회귀를 모두 푸는 지도학습 모델



결정 트리를 배우는 이유 규칙이 눈에 보여 이해하기 쉽고, 랜덤 포레스트·XGBoost·LightGBM 같은 강력한 모델의 기본 부품이다.

ⓒ 2026. DJ all rights reserved.

02

결정 트리

예/아니오 질문을 이어 가며 답을 찾는 모델

02 결정 트리(Decision Tree)란?

질문을 이어 가며 결정을 내리는 나무 모양의 모델

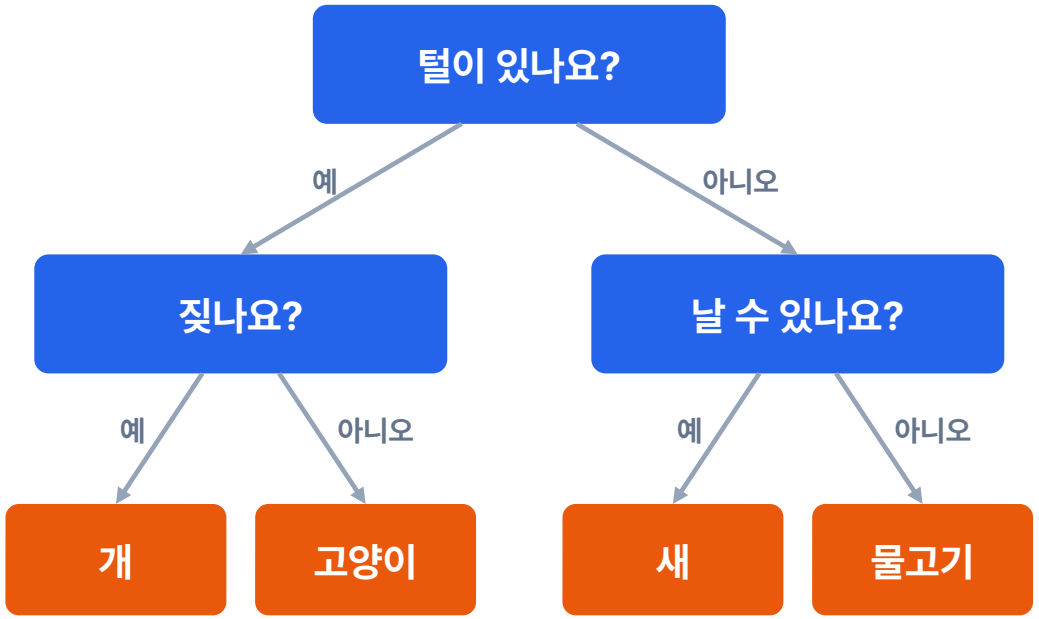
결정 트리란

- 여러 선택 사항과 결과를 나무 모양 그래프로 나타낸 결정 도구
- '만약 ~라면, 그다음은?' 질문의 연속
- 결정 과정을 단계별로 보여 주어 복잡한 문제도 이해하기 쉽다

머신러닝에서는

- 데이터를 보고 예/아니오 질문 목록을 스스로 학습한다
- 정답에 가장 빨리 닿는 질문을 고른다
- 이해하기 쉽고 그림으로 보여 주기도 쉬워 널리 쓰인다

예: 동물 맞히기 게임

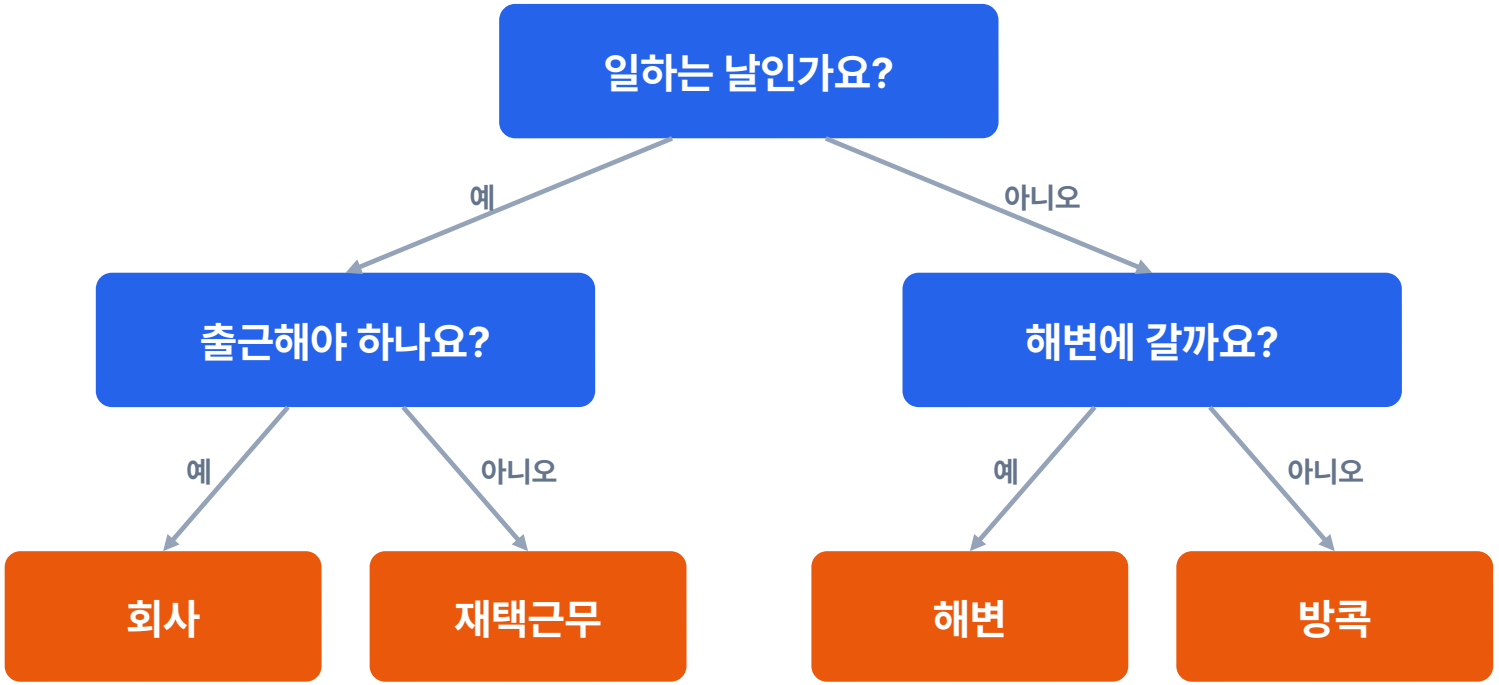


파란 상자 = 질문(노드) · 주황 상자 = 최종 답(리프)

ⓒ 2026. DJ all rights reserved.

02 결정 트리 예시: 오늘 어디로 갈까?

네 가지 선택(회사 · 재택근무 · 해변 · 방콕)을 질문 두 번으로 정한다



분류 문제에 대한 이진 결정 트리 (질문마다 답이 예/아니오 두 가지)

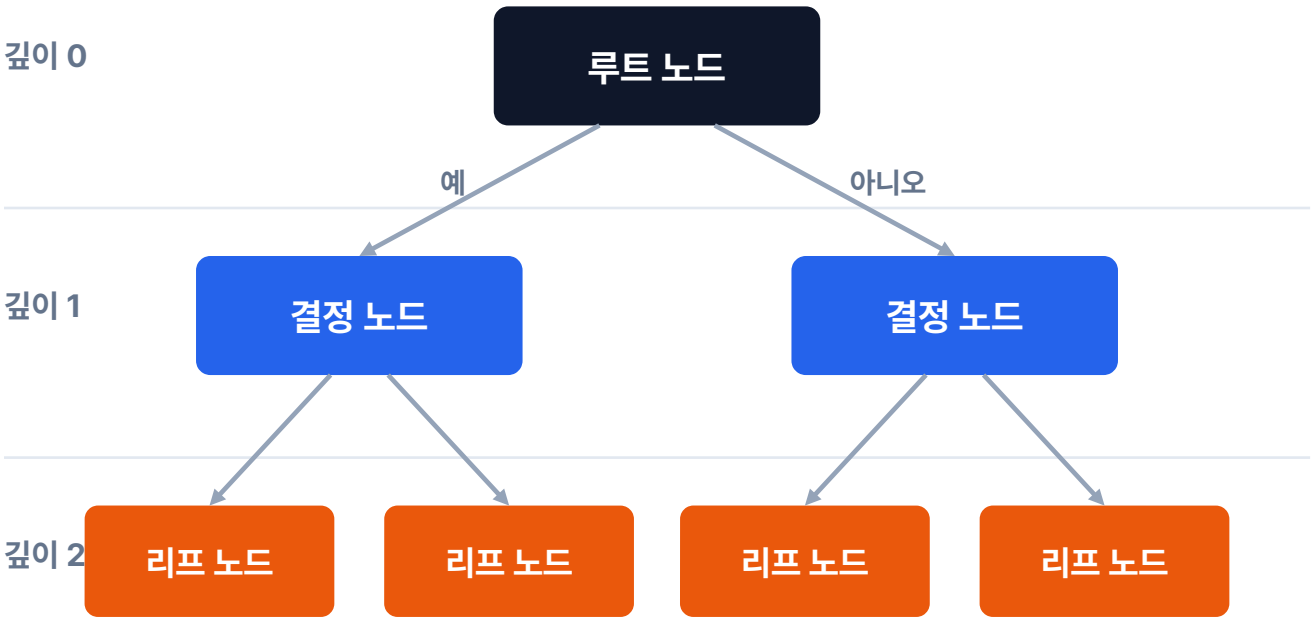
왜 트리로 만들까?

- 선택지가 네 개라 매번 같은 고민을 한다
- 그동안 내가 했던 선택(데이터)을 토대로 트리를 만들어 두면
- 다음부터는 트리를 따라 빠르게 결정할 수 있다

ⓒ 2026. DJ all rights reserved.

02 결정 트리의 기본 용어

루트 노드 · 결정 노드 · 리프 노드 · 깊이



화살표 하나하나를 '가지(branch)'라고 한다

루트 노드 (root)
맨 위의 첫 질문

결정 노드 (decision)
중간에서 다시 질문하는 노드

리프 노드 (leaf)
더 나뉘지 않는 맨 끝 노드. 여기서 예측값이 나온다

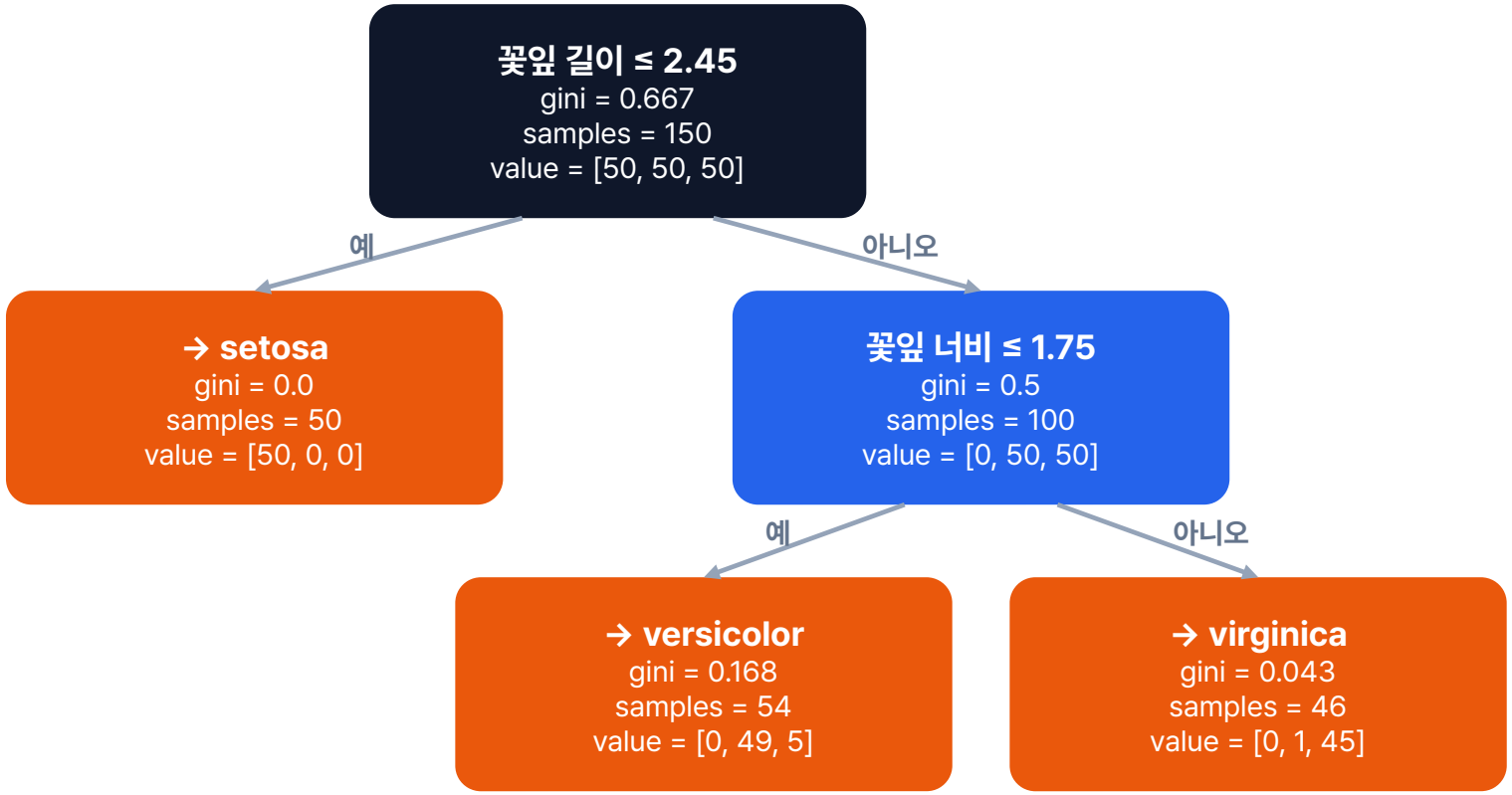
깊이 (depth)
루트에서 몇 번 나뉘었는지. 루트는 0, 이 트리는 깊이 2

순수 노드
한 클래스의 샘플만 남은 노드 (지니 불순도 = 0)

ⓒ 2026. DJ all rights reserved.

02 분류 트리: 붓꽃 예시

max_depth=2로 학습한 실제 트리 (scikit-learn 1.9, 붓꽃 150송이)



읽는 법

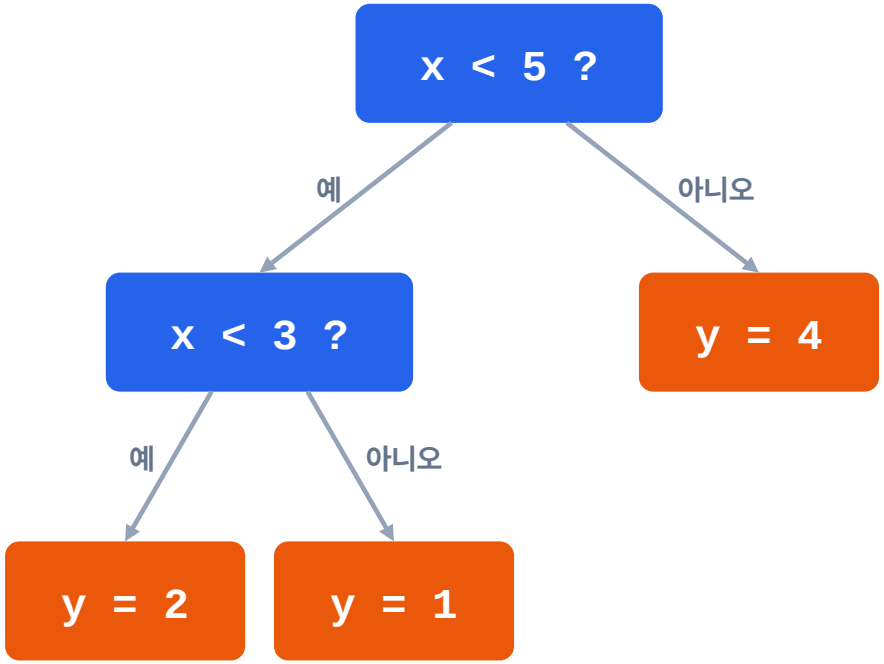
- gini: 노드의 불순도 (0이면 한 클래스만)
- samples: 이 노드에 도착한 샘플 수
- value: 클래스별 샘플 수 [setosa, versicolor, virginica]
- 리프에서 가장 많은 클래스로 예측한다
- 학습 데이터 정확도 0.96

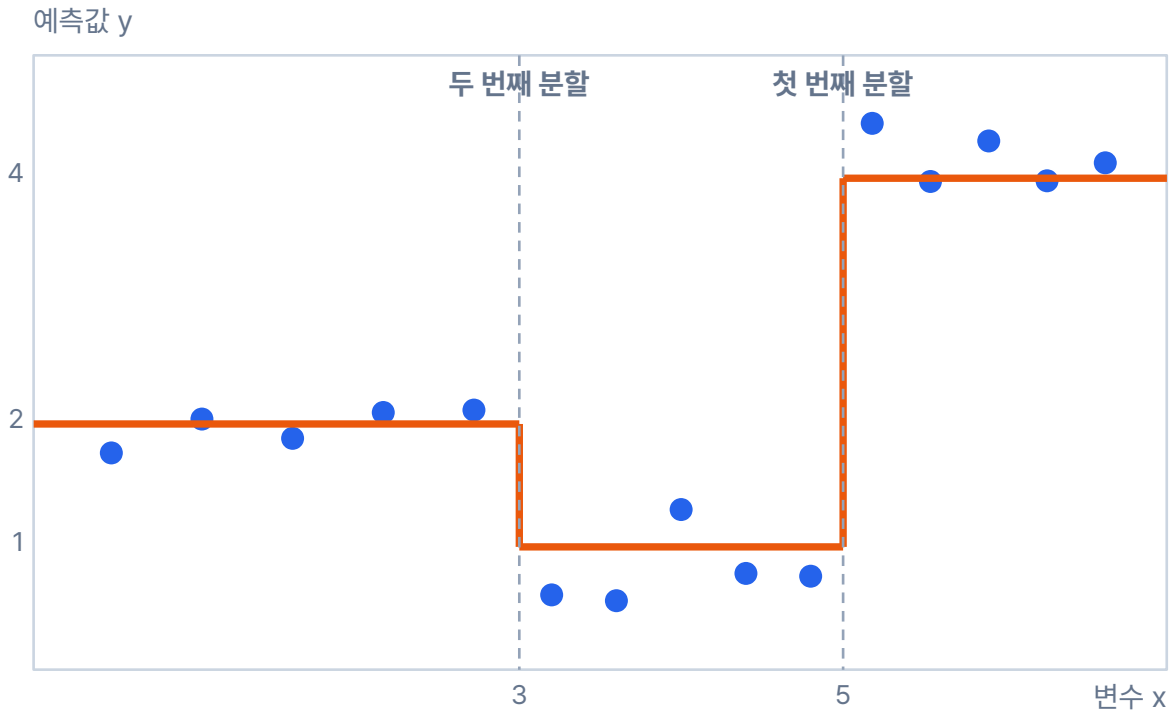
```
tree = DecisionTreeClassifier(max_depth=2, random_state=42).fit(iris.data, iris.target)
plot_tree(tree, filled=True)      # 그림으로 보기          print(export_text(tree))   # 글자로 보기
```

ⓒ 2026. DJ all rights reserved.

02 회귀 트리: 숫자 예측

리프에 모인 샘플의 평균값으로 예측한다





분할 기준 평균제곱오차(MSE)가 가장 많이 줄어드는 곳에서 나눈다. **`criterion="squared_error"`** (기본값, 예전 이름 "mse"는 없어졌다)

ⓒ 2026. DJ all rights reserved.

02 가지치기(Pruning)

결정 트리는 과대적합되기 쉬워, 너무 자라지 않게 가지를 친다

(A) 사전 가지치기 (Pre-pruning)

- 트리가 다 자라기 전에 학습을 멈춘다
- 깊이, 리프 개수, 리프의 최소 샘플 수 등을 제한한다

```
DecisionTreeClassifier(max_depth=3,
                       min_samples_leaf=5)
```

(B) 사후 가지치기 (Post-pruning)

- 트리를 끝까지 키운 뒤, 아래에서부터 가지를 쳐 나간다
- scikit-learn: 비용 복잡도 가지치기 ccp_alpha

```
path = tree.cost_complexity_pruning_path(X, y)
DecisionTreeClassifier(ccp_alpha=0.005)
```

예 유방암 데이터에서 끝까지 키운 트리는 테스트 정확도 0.951, ccp_alpha≈0.0046으로 가지를 치면 0.958

ⓒ 2026. DJ all rights reserved.

03

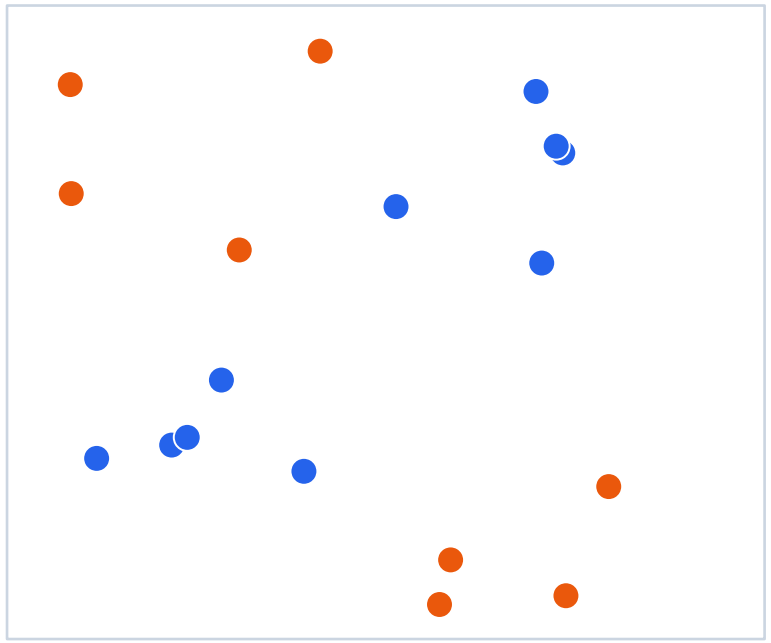
결정 트리의 결정 경계

질문 하나가 공간을 직선으로 한 번씩 자른다

03 결정 트리의 결정 경계(decision boundary)

분할할 때마다 특성 축에 수직인 경계선이 하나씩 생긴다

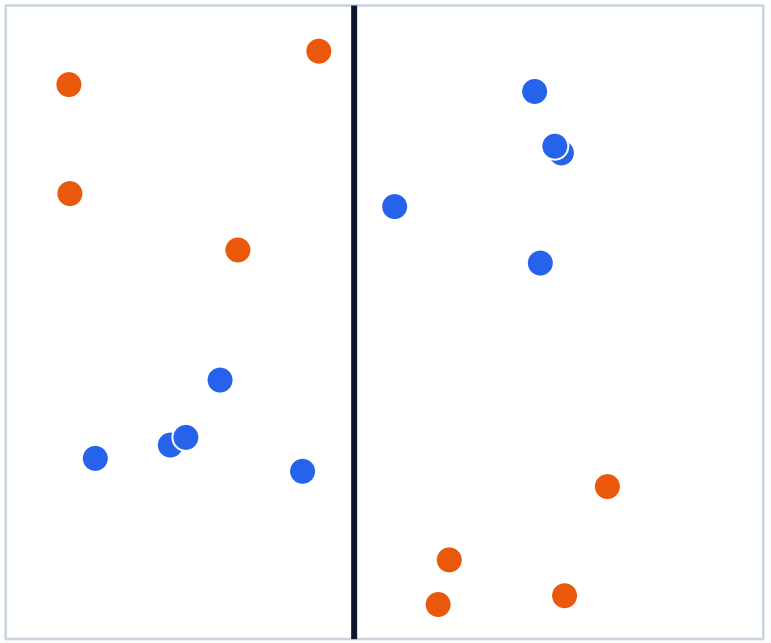
깊이 0 나누기 전: 영역 하나



↑ 세로축: 특성2 (평균 연령)

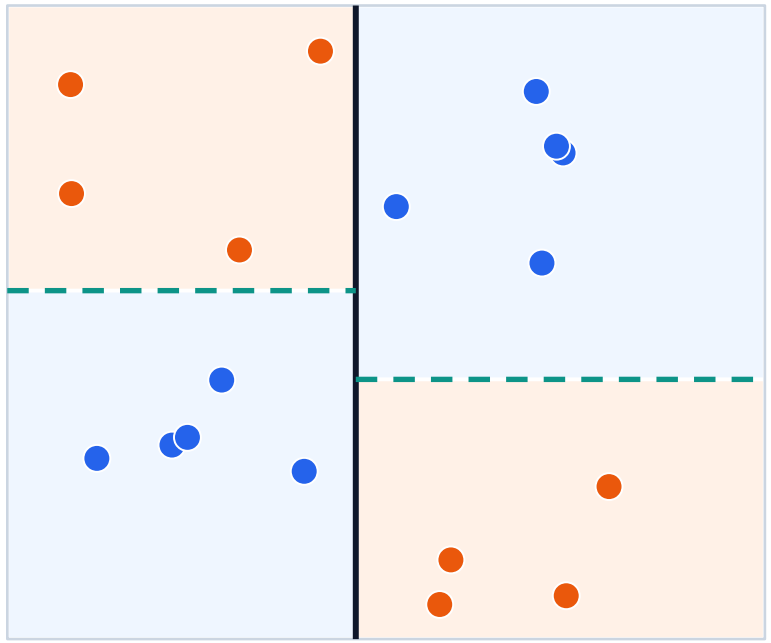
특성1 (평균 수입)

깊이 1 평균 수입으로 한 번 → 영역 2개



특성1 (평균 수입)

깊이 2 각 영역을 평균 연령으로 → 영역 4개



특성1 (평균 수입)

결정 트리의 경계는 **항상 축에 평행한 직선 조각**으로 이루어진다. 깊어질수록 영역이 잘게 나뉘어 학습 데이터에 딱 맞게 된다(과대적합 위험).

ⓒ 2026. DJ all rights reserved.

04

지니 불순도와 엔트로피

어떤 질문으로 나눌지 정하는 기준

04 분할 기준: 불순도(impurity)

노드 안이 얼마나 섞여 있는지 재서, 가장 깨끗하게 나누는 질문을 고른다

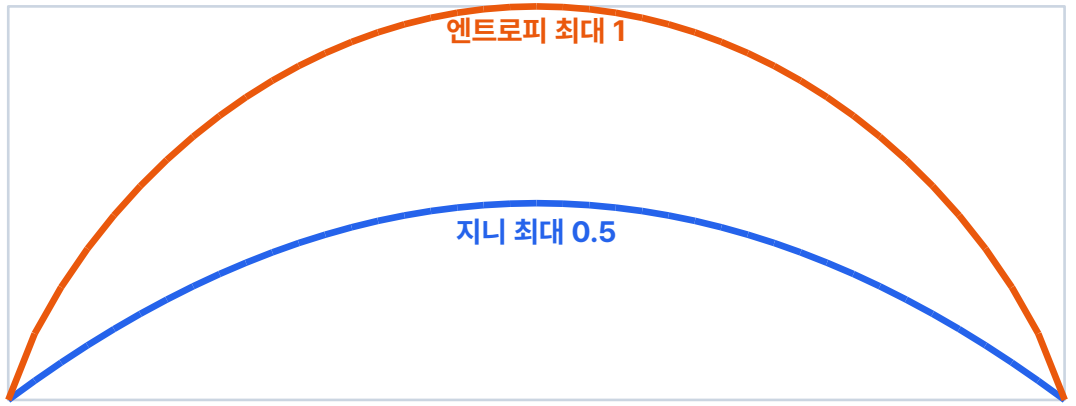
지니 불순도 (Gini impurity)

- $G = 1 - \Sigma\, p_i^2$ (p_i: 노드에서 클래스 i의 비율)
- 0이면 한 클래스만(순수), 두 클래스가 반반이면 0.5로 가장 크다
- 경제학의 '지니계수'와 이름은 비슷하지만 다른 개념이다

엔트로피 (Entropy)

- $H = -\Sigma\, p_i \log_2 p_i$
- 정보 이론에서 불확실성(무질서)을 재는 값. 열역학의 개념에서 왔다
- 0이면 순수, 두 클래스가 반반이면 1로 가장 크다

불순도 (두 클래스일 때)



criterion	분류	회귀
기본값	`"gini"`	`"squared_error"`
다른 값	`"entropy"` `"log_loss"`	`"absolute_error"` `"poisson"`

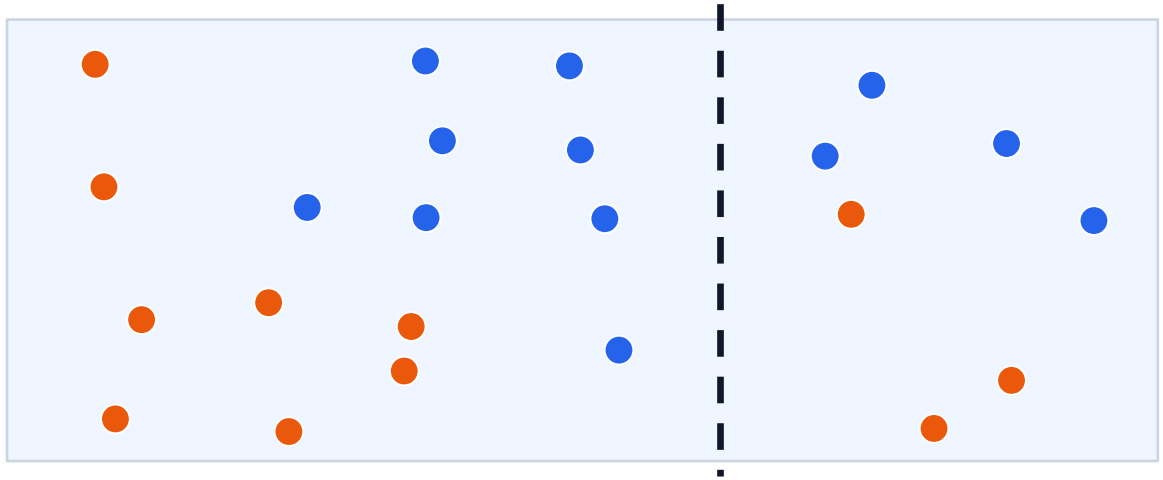
"log_loss"는 "entropy"와 같다. 정보 이득(IG)은 별도 불순도가 아니라 "분할 전 − 분할 후" 감소량이다.

ⓒ 2026. DJ all rights reserved.

04 지니 불순도 계산 예시

두 가지 분할 A · B 중 지니가 더 낮은 쪽을 고른다 (파랑 12개 · 주황 11개)

분할 A (세로로 나누기)

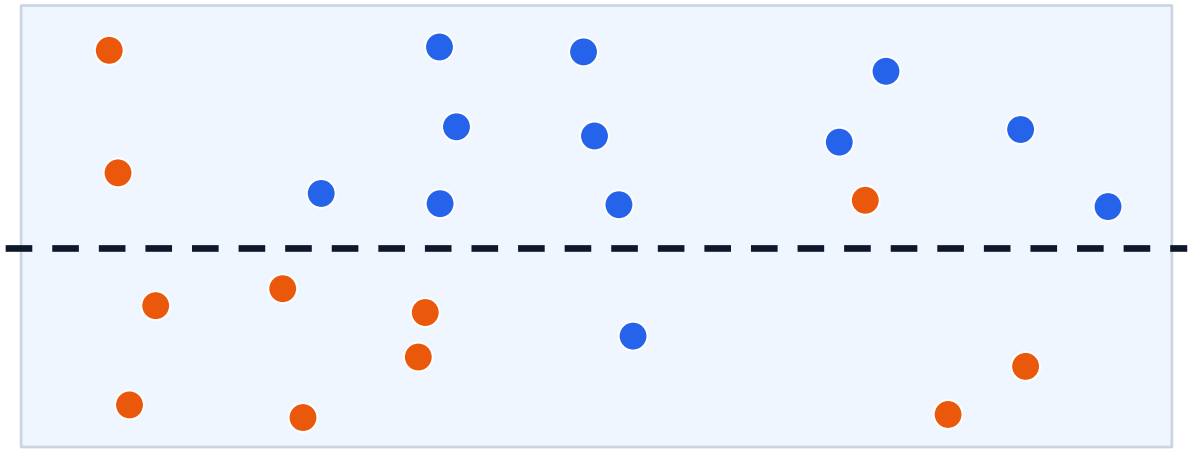


왼쪽 16개 (파랑 8 · 주황 8): $1 - (8/16)^2 - (8/16)^2 = 0.50$

오른쪽 7개 (파랑 4 · 주황 3): $1 - (4/7)^2 - (3/7)^2 = 0.49$

가중 평균 $16/23 \times 0.50 + 7/23 \times 0.49 = 0.497$

분할 B (가로로 나누기)



위 14개 (파랑 11 · 주황 3): $1 - (11/14)^2 - (3/14)^2 = 0.34$

아래 9개 (파랑 1 · 주황 8): $1 - (1/9)^2 - (8/9)^2 = 0.20$

가중 평균 $14/23 \times 0.34 + 9/23 \times 0.20 = 0.28$

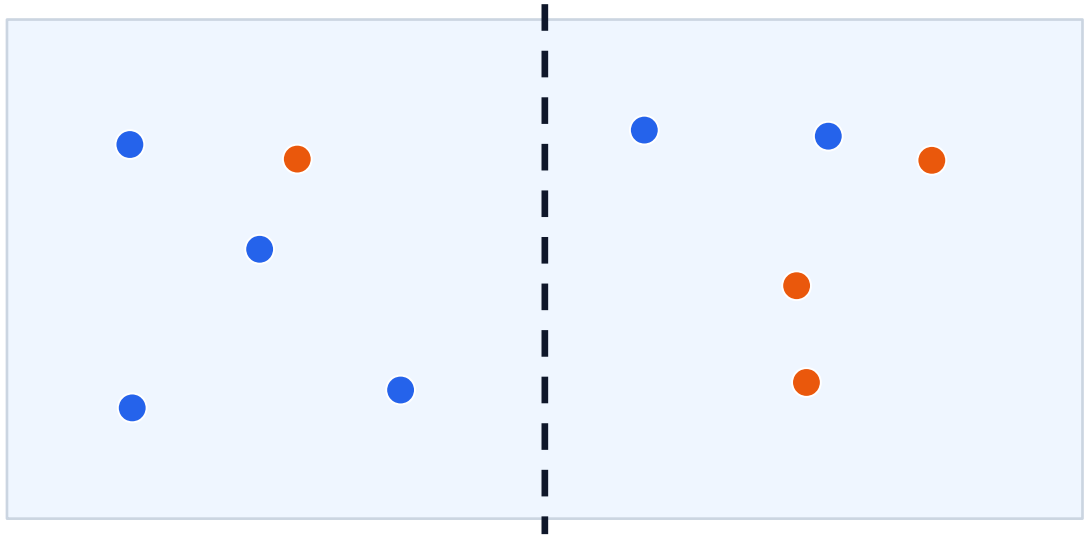
결론 나누기 전 지니 0.499 → A 0.497(거의 그대로) vs B 0.28(크게 감소) → 트리는 **B로 나눈다**

ⓒ 2026. DJ all rights reserved.

04 엔트로피 계산과 정보 이득

분할 뒤 엔트로피가 얼마나 줄었는지(정보 이득)로 질문을 고른다

10개 (파랑 6 · 주황 4)를 세로로 나누기



왼쪽 5개: 파랑 4 · 주황 1　　오른쪽 5개: 파랑 2 · 주황 3

분할 전　$-(6/10)\cdot\log_2(6/10) - (4/10)\cdot\log_2(4/10) = 0.971$

왼쪽　$-(4/5)\cdot\log_2(4/5) - (1/5)\cdot\log_2(1/5) = 0.722$

오른쪽　$-(2/5)\cdot\log_2(2/5) - (3/5)\cdot\log_2(3/5) = 0.971$

분할 후　$0.5 \times 0.722 + 0.5 \times 0.971 = 0.846$



정보 이득 = 0.971 − 0.846 = 0.125

정보 이득이 가장 큰 분할을 고른다 (주황 막대가 줄어든 만큼)

지니를 쓰면 **지니 감소량**, 엔트로피를 쓰면 **정보 이득**이 가장 큰 질문을 고른다. 둘의 결과는 대부분 비슷하다.

ⓒ 2026. DJ all rights reserved.

05

장단점과 매개변수

트리의 복잡도를 조절해 과대적합을 막는다

05 모델 복잡도를 조절하는 매개변수

트리가 얼마나 크게 자랄지 정한다 (하이퍼파라미터)

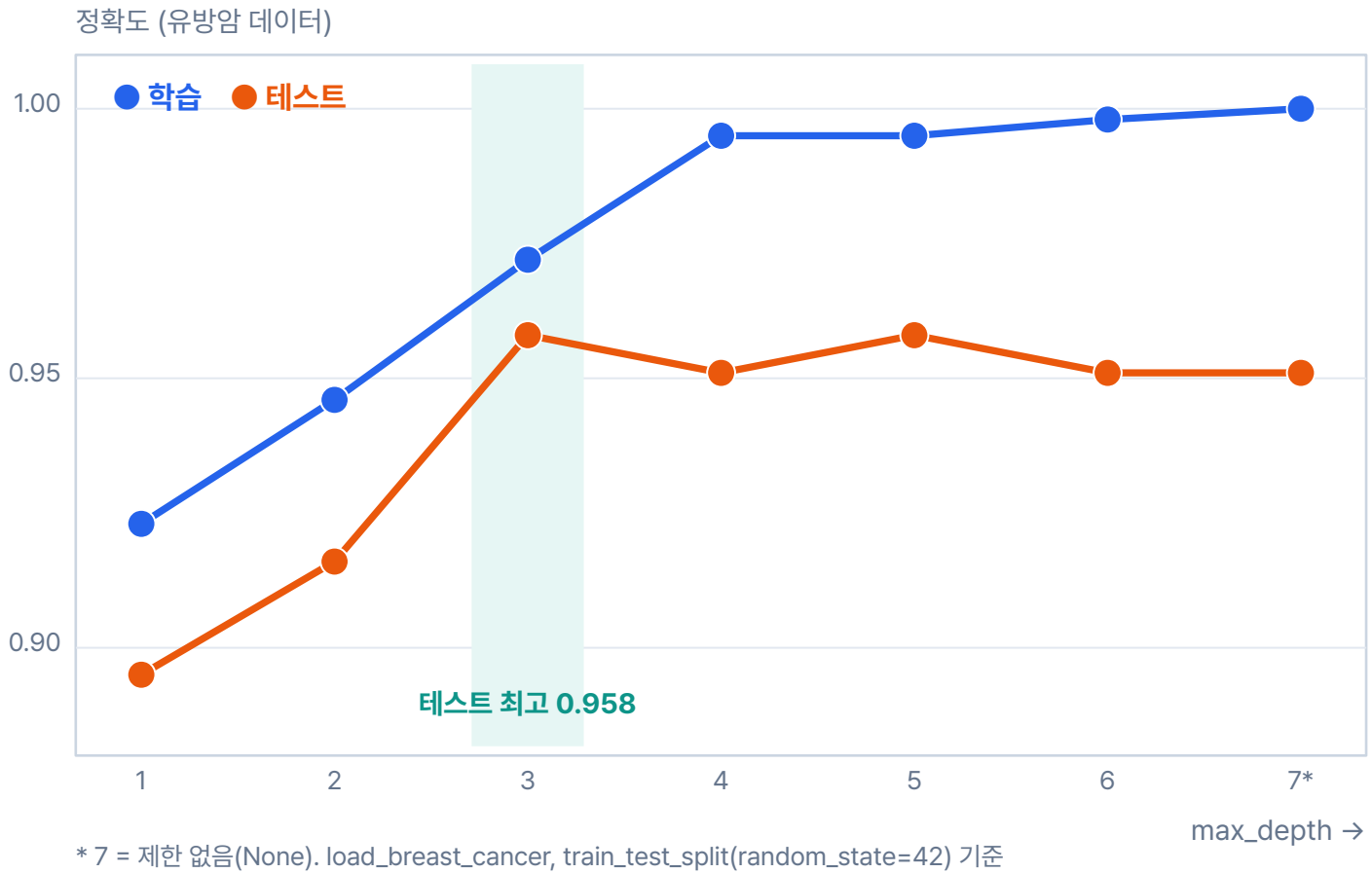
매개변수	뜻	값을 바꾸면
`max_depth`	트리의 최대 깊이	작을수록 단순해진다
`max_leaf_nodes`	리프 노드의 최대 개수	작을수록 단순해진다
`min_samples_split`	노드를 나누려면 있어야 하는 최소 샘플 수	클수록 단순해진다
`min_samples_leaf`	리프 노드가 가져야 하는 최소 샘플 수	클수록 단순해진다
`ccp_alpha`	사후 가지치기의 강도 (0이면 가지치기 안 함)	클수록 많이 잘라 낸다

```
DecisionTreeClassifier(max_depth=3, min_samples_leaf=5, random_state=42)
```

ⓒ 2026. DJ all rights reserved.

05 깊이와 과대적합

깊어질수록 학습 정확도는 1.0에 가까워지지만, 테스트 정확도는 어느 순간 멈춘다



* 7 = 제한 없음(None). load_breast_cancer, train_test_split(random_state=42) 기준

읽는 법

- 깊이 3 근처에서 테스트 정확도가 가장 높다 (0.958)
- 그 뒤로는 학습 정확도만 오르고 테스트는 제자리다 → 과대적합
- max_depth를 정하지 않으면 깊이 7, 리프 16개까지 자라 학습 데이터를 다 외운다 (학습 정확도 1.0)

ⓒ 2026. DJ all rights reserved.

05 결정 트리의 장단점

이해하기 쉽지만 과대적합되기 쉽다

장점

- 만든 모델을 그림으로 보여 줄 수 있어 비전문가도 이해하기 쉽다
- 특성을 하나씩 따로 보고 나누므로 데이터 스케일의 영향을 받지 않는다 → 정규화·표준화 같은 전처리가 필요 없다
- 특성의 범위가 서로 다르거나, 이진 특성과 연속 특성이 섞여 있어도 잘 작동한다
- 결측값이 있어도 바로 학습할 수 있다 (scikit-learn 1.3+)

단점

- 과대적합(overfitting)되는 경향이 있다 → 대안으로 앙상블(랜덤 포레스트, 그래디언트 부스팅)을 쓴다
- 데이터가 조금만 바뀌어도 트리 모양이 크게 바뀔 수 있다
- 회귀 트리는 학습 데이터 범위 밖의 값을 예측하지 못한다 (리프 평균만 내놓는다)

ⓒ 2026. DJ all rights reserved.

06

앙상블 기법

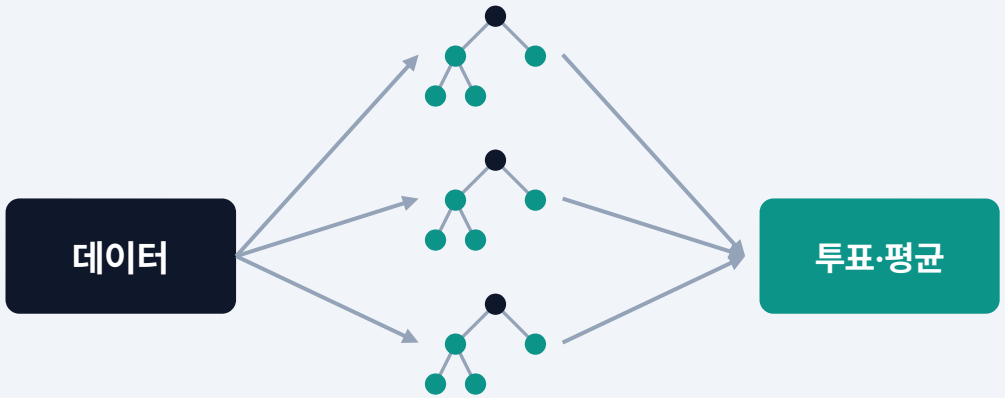
여러 모델을 묶어 더 강한 모델을 만든다

06 앙상블(Ensemble)이란?

여러 머신러닝 모델을 연결해 더 강력한 모델을 만드는 기법

배깅 (Bagging)

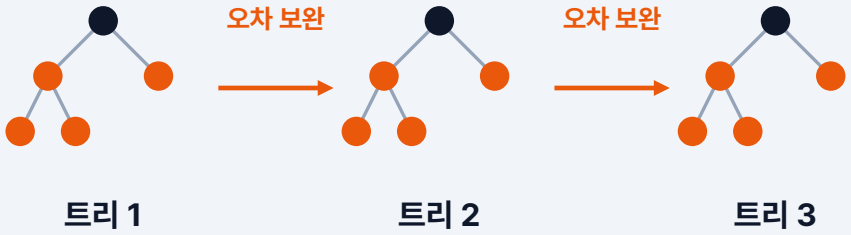
대표: 랜덤 포레스트



트리를 따로따로(병렬) 만들고 결과를 모은다

부스팅 (Boosting)

대표: 그래디언트 부스팅



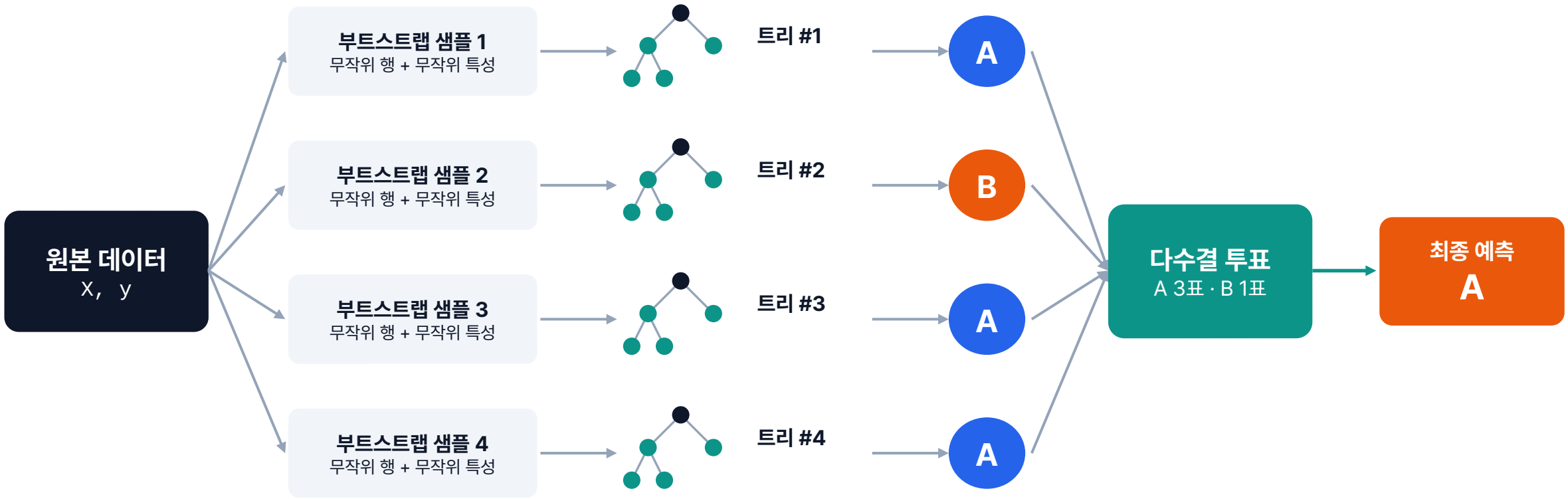
앞 트리가 틀린 부분을 다음 트리가 이어서(직렬) 고친다

둘 다 기본 부품으로 결정 트리를 쓴다. 부스팅 라이브러리: XGBoost · LightGBM · CatBoost · scikit-learn HistGradientBoosting (앙상블 자료에서 자세히)

ⓒ 2026. DJ all rights reserved.

06 랜덤 포레스트의 아이디어

조금씩 다른 결정 트리를 많이 만들어 투표한다



분류는 **다수결 투표**, 회귀는 **트리 예측의 평균**으로 최종 예측을 낸다.

ⓒ 2026. DJ all rights reserved.

06 랜덤 포레스트의 원리

잘 맞히면서도 서로 다른 트리를 많이 만들면, 평균할 때 과대적합이 줄어든다

좋은 트리의 조건

- (1) 타깃을 잘 예측해야 한다
- (2) 다른 트리와 달라야 한다

각 트리는 과대적합돼 있어도 서로 다른 방향으로 틀리기 때문에, 평균(투표)하면 오차가 줄어든다.

A 데이터 포인트를 무작위로 고른다

부트스트랩 샘플: 중복을 허용해 원래 크기만큼 뽑는다 (bootstrap=True)

원본	1	2	3	4	5	
샘플	**2**	**2**	4	5	1	**2가 두 번, 3은 빠짐**

B 특성(feature)을 무작위로 고른다

노드를 나눌 때마다 일부 특성(max_features)만 후보로 본다

후보	꽃받침 길이	**꽃받침 너비**	**꽃잎 길이**	꽃잎 너비

이번 노드에서는 주황 두 개만 보고 질문을 고른다

v12의 '수학적으로 증명됨'은 **여러 트리의 예측을 평균하면 분산(들쭉날쭉함)이 줄어든다**는 배깅의 원리를 말한다.

ⓒ 2026. DJ all rights reserved.

07

랜덤 포레스트

주요 매개변수 · 장단점 · 실습 코드

07 랜덤 포레스트의 주요 매개변수

scikit-learn RandomForestClassifier / RandomForestRegressor 기준

매개변수	뜻	기본값
`n_estimators`	만들 트리의 개수	100
`max_features`	노드를 나눌 때 후보로 볼 특성 수	분류 "sqrt" · 회귀 1.0 (전체)
`max_depth` 등	각 트리의 복잡도 (결정 트리와 같음)	None (끝까지 자람)
`n_jobs`	학습에 쓸 CPU 코어 수 (-1이면 전부)	None (1개)
`random_state`	결과를 다시 똑같이 얻기 위한 난수 고정	None

n_estimators는 클수록 좋을까? **장점** 더 많은 트리를 평균하면 과대적합이 줄어든다. **단점** 메모리와 학습 시간이 늘고, 어느 정도를 넘으면 성능이 더 오르지 않는다.

ⓒ 2026. DJ all rights reserved.

07 랜덤 포레스트의 장단점

튜닝을 많이 하지 않아도 성능이 좋다

장점

- 성능이 매우 뛰어나고, 매개변수 튜닝을 많이 하지 않아도 잘 작동한다
- 데이터의 스케일을 맞출 필요가 없다
- 트리를 병렬로 만들 수 있다 (n_jobs)

단점

- 텍스트처럼 차원이 매우 높고 희소한 데이터에서는 잘 작동하지 않는다
- 선형 모델보다 메모리를 많이 쓰고 예측이 느리다
- 트리가 많아 트리 하나처럼 그림으로 설명하기는 어렵다

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, random_state=42).fit(X_train, y_train)
rf.score(X_test, y_test)      # 0.965   (같은 데이터에서 결정 트리 하나는 0.951)
```

ⓒ 2026. DJ all rights reserved.

정리 한 장으로 보는 결정 트리

결정 트리

예/아니오 질문을 이어 가며 예측한다.

분류는 리프의 다수 클래스, 회귀는 리프의 평균.

분할 기준

분류: 지니 불순도·엔트로피, 회귀: 제곱 오차.

불순도가 가장 많이 줄어드는 질문을 고른다.

과대적합 막기

max_depth·min_samples_leaf로 미리 멈추거나(사전),

ccp_alpha로 나중에 잘라 낸다(사후).

앙상블

랜덤 포레스트: 데이터·특성을 무작위로 고른 트리들의 투표.

부스팅: 앞 트리의 오차를 다음 트리가 이어서 보완.

REF 참고 자료

더 공부하고 싶다면

참고 자료

- 사전·사후 가지치기 — lucy-the-marketer.kr/ko/growth/pruning-decision-tree-to-prevent-overfitting/
- 의사결정나무 CART 알고리즘과 지니계수 — leedakyeong.tistory.com
- 랜덤 포레스트 그림 참고 — bitly.kr/s2GOAgZm0
- scikit-learn 공식 문서: 결정 트리 — scikit-learn.org/stable/modules/tree.html
- scikit-learn 공식 문서: 앙상블 — scikit-learn.org/stable/modules/ensemble.html

© 2026. DJ all rights reserved.